

A Virtual Keyboard Implementation Based on Finger Recognition

Y. Zhang, W. Yan and A. Narayanan
Auckland University of Technology, 1010, New Zealand

Abstract— Keyboards still remain as the most popular input medium of choice for users needing to input large amounts of error-free text or combinations of text and numeric data. As mobile devices are being designed smaller, it becomes increasingly difficult for users to enter large amounts of input without carrying and attaching a keyboard. In this paper, we develop a new type of virtual keyboard that allows users to type on any plane to any device. The virtual keyboard is customized and printed on plain paper so that it can be put on any oblique plane or stuck on a wall. The device camera is then used for key recognition based on hand skin tone and fingertip location. If the fingertip remains on a key for a predetermined amount of time, the program will regard this key as an input. Our experiments show how various customized virtual keyboards allow users to input text of their choice if they have a real keyboard in front of them and with no identifiable performance compromise.

Keywords—*human-computer interface; virtual keyboard; finger recognition*

I. INTRODUCTION

Mobile devices continue to develop and gain prominence with the popular press increasingly predicting the end of PCs. Obvious drawbacks to this scenario, including lack of keyboard input and small screens that make it difficult to present complex graphs and full width text, tend to be dismissed by referring to docking stations which presumably have to be carried around with the device, because of lack of technical compatibility between tablet and smartphone manufacturers. However, users are also not keen to carry around laptops and PCs because of the weight and possible damage in transit if they have the option of using mobile devices. An ideal solution for such users is portability with rich input and output media without the need to bring any other equipment with them or rely on available docking stations and peripherals.

The main advantages of smartphones and other mobile devices like iPads are that they are portable and convenient because the screens are also the interface for human-computer interaction (HCI) [15, 26, 40]. Although a touch screen is effective and portable [5], it also has limitations because the screen is covered by the user's hand and smeared by fingers, food, oil, ink, etc. Additionally, the screen will inevitably become worn out due to the frictions caused by touching the screen. Finally, it is impossible for users to type by using multiple fingers on a small touch screen [25]. Unless these obstacles can be overcome, computers and laptops will remain popular despite being relatively big, heavy and cumbersome.

Unlike smartphones and iPads, desktops and laptops make use of a keyboard to get human-computer interaction (although touch-screen lap-tops and PCs are becoming more popular, especially because of the prevalence of Apps) [5, 31, 40]. As a common and classic human-computer interface, the keyboard is not very portable but users still generally accept it. Until reliable natural language interfaces become available, keyboards will remain the dominant interface for text input.

Even if natural language interfaces become sufficiently reliable, it is easy to imagine that there will still be a need for keyboard input. [28]

A keyboard also requires a great deal of resources such as the special costs of materials, electricity, and cables. In addition, a normal keyboard is restricted by its plane, its fixed-key layout can be easily damaged. Keyboards will inevitably end up as rubbish when they stop working, which could cause environmental pollution. Interest has grown recently regarding alternative keyboard technologies that overcome these limitations.

In this paper, we develop a new type of virtual keyboard that has advantages of current physical keyboards but which overcomes many of the problems. We will introduce our related work in Section 2, our framework will be stated as Section 3, and the prototype implementation will be addressed in Section 4, we will discuss our results in Section 5, in Section 6 we will identify limitations and present future work.

II. RELATED WORK

The concept of a virtual keyboard is that it has the same input and output functionalities as a traditional mechanical keyboard but is not mechanical. Several technologies such as digital image processing, pattern recognition, finger recognition, and tracking are required to complete a virtual keyboard system [8]. Currently, there are still a variety of challenges that must be overcome to develop a robust virtual keyboard. These challenges include variable conditions such as differences in skin tones between users, complex environments, inconvenient add-on devices, and the limitations of various algorithms [9].

3D hand models are used to capture the complex motion of human hand gesture [14] whilst hand gesture recognition is accomplished mainly through static hand posture recognition and temporal gesture recognition [2, 3, 4]. However, there is still much room for improvement in current hand gesture recognition systems [22]. A continuous posture recognition system was developed based on a recurrent neural network (RNN) to deal with dynamic gesture processes in real time [10, 17, 29, 37, 41]. The proposed gesture recognition system can recognize Japanese finger alphabets including 42 characters [16, 17, 29, 37]. In the experiments, the recognition rate increased through adding learning patterns. However, the data was sampled by gloves covered with reflective markers; hence, the method is not convenient in reality.

Another approach is to use images captured by a high-frame-rate camera to track 3D motion of the human hand and fingers so that in-air typing motion can be recognized [19, 20, 23, 32]. Overall, the objective of such systems is to implement a fast finger tracking system that can be used as an in-air typing interface. By using a template-matching-based method, such systems can significantly reduce background effect and make

tracking possible even without markers. High frame rate cameras are used because they are capable of tracking and recognizing rapid typing motion [39]. Furthermore, real-time recognition can be accomplished if the corresponding hardware is configured for the purpose of parallelizing and accelerating the image processing required [24, 36, 38]. Several systems have been implemented using this approach of motion recognition of in-air typing in real time [10, 11, 13, 20, 21]. Also, these template matching-based methods proved effective even when faced with environments of varying illumination and cluttered backgrounds. However, these systems require appropriate capture hardware and high online processing power, none of which is available with mobile devices currently.

A depth-based hand gesture recognition system [34] has been developed that can be used to control household appliances [30, 34]. The system makes use of a depth camera and achieves recognition of dynamic hand gestures by using static posture recognition and hand trajectories [6]. In static gesture recognition, the features of the hand are extracted and used to classify the hand posture [35]. Another method was utilized to a depth camera to capture 3D hand motion trajectory [27]. The method made use of dynamic time warping (DTW) algorithms [7] combined with k-NN classification. This combination was chosen because it could be implemented with ease and adapt to different users and varying gesture types. The generalizability of these approaches to virtual keyboards is not clear.

Operating areas on touch screens are becoming smaller with the miniaturization of mobile devices. Consequently, the much narrower screen means that the built-in virtual keyboard occupies a larger portion of the screen. Furthermore, the overall smaller size of the screen can make it uncomfortable for users to type on. An in-air typing interface was developed [20] to remove these disadvantages of traditional mobile interfaces. The interface requires add-on devices including a high framerate camera, an IR filter, four IR LEDs, and a vibration motor [39]. Lucas-Kanade algorithm was combined with a band-pass filter and used to recognize and track the user's fingertip. Two experiments were designed to test its usability of the interfaces based on three aspects - frame rate, latency, and tactile feedback. Based on these results, reducing several milliseconds of latency shortens task completion time. In order to control the latency, the frame rate should not be lower than 15 FPS (frames per second). Unexpectedly, mistypes decreased significantly thanks to vibration feedback.

The main limitation of in-air typing approaches is the need for a reference interface or template so that the users can obtain visual information concerning location of fingers for accurate typing. Having to look at the output screen as well as having to constantly adjust to the reference template so as to ensure that fingers are still accurately positioned on the keys is more complex than using a keyboard alone. If the interface or reference template is also hand-held, only one hand is available for text input [12], which compromises the idea of genuine in-air typing using fingers of both hands. Also, the problems associated with single-handed typing need to be identified and resolved first before adapting the technology to two-handed typing [14, 20]. Another limitation is the need for high

performance hardware to implement many of these template-based approaches in real time and with accuracy.

In this paper, we will demonstrate the feasibility of a novel approach to virtual keyboarding that minimizes the requirement for extra hardware for single-handed typing, and evaluate its performance. In the conclusion, we will discuss limitations and further work required to enhance the performance of technology.

III. OUR FRAMEWORK

The input resources of our framework of this paper are a customized keyboard and a video camera, such as a smartphone, that records the movement of a user's finger as the user enters letters and numbers on the keyboard [39]. In the implementation of our virtual keyboard, there are two subsystems that are responsible for processing the input customized keyboards and videos. These two subsystems are for fingertip recognition and the recognition of the customized keyboard, respectively.

In the subsystem of fingertip recognition, a Gaussian filter smooths each frame of the input video before the image becomes dilated and eroded. The filtered image is split into the three R, G, B channels and then binarized.

Image binarization divides all pixels of a grayscale images into two groups according to a certain threshold and then assigns 0 and 255 to pixels in these two groups, respectively. Supposed that the function of an input image is $f(x, y)$ and the function of the output image is $g(x, y)$ as Eq. (1).

$$g(x, y) = \begin{cases} 0 & f(x, y) < T \\ 255 & f(x, y) \geq T \end{cases} \quad (1)$$

where the threshold $T \geq 0$ is a scale plate that separates foreground and background regions. Hence, a proper threshold is able to not only maintain the useful information of an image but also decrease the interference of background and noise as much as possible.

The BWMORP algorithm as implemented in Matlab is utilized to extract a skeletal image of the human hand and then the furthest endpoint of the skeleton is selected to represent the fingertip [18]. An example of this procedure is shown in Figure 1. When this fingertip is detected, the coordinates of the pixel are known as well.

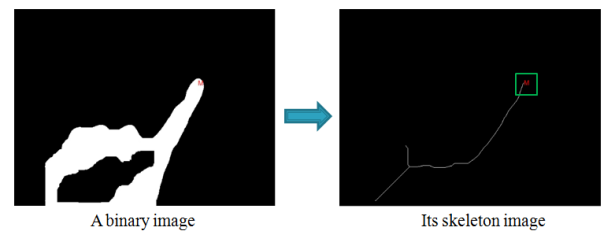


Figure 1. The BWMORP algorithm for skeleton extraction

To meet the needs of virtual keyboard customization, the OCR tool in Matlab was utilized to extract all the characters and numbers of the paper keyboard image. We convert the input keyboard image to a binary one and smooth it to decrease

noises. Meanwhile, slant distortions are also rectified and the characters are segmented. Finally, the preprocessed characters are recognized [35].

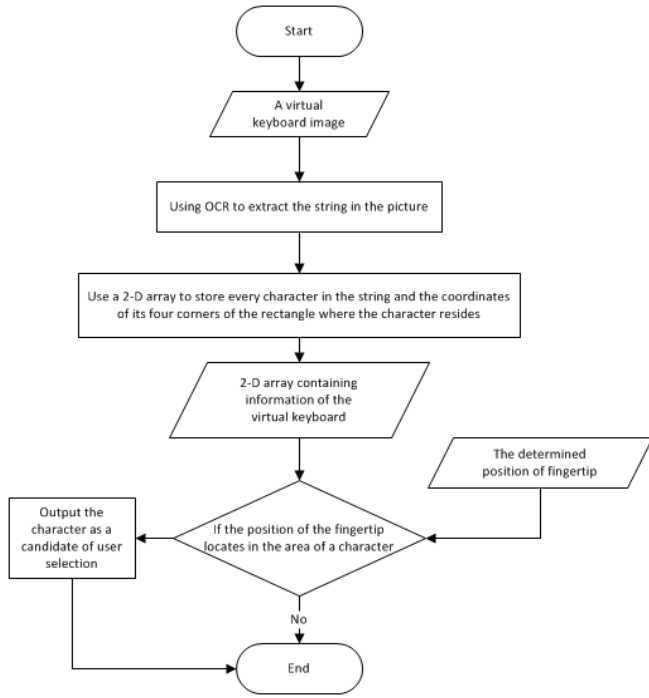


Figure 2. The flowchart of the customized keyboard recognition

A 2D array then stores all the characters and numbers with their coordinates related to the four corners of the rectangle in which they are located. The 2D array effectively locates the virtual keyboard that the program will read. Coordinates of the fingertip are given by the output of fingertip recognition so that when the fingertip locates in the area of a key, the system will record this key as a candidate that the user might want to type. The flowchart of the customized keyboard recognition is shown in Figure 2.

The customized keyboard recognition produces a computer-readable matrix for various customized virtual keyboards. The fingertip recognition will keep detecting positions of the fingertip in each frame to track the fingertip on the virtual keyboard. The fingertip moves on the virtual keyboard, and when the fingertip stays on a key for more than the duration of four frames, the computer will treat this motion as a typing action executed by the user. The use of four frames was determined by experiments and this was the number that caused the least errors. According to the coordinates of the point where the fingertip stayed, the corresponding key is found on the virtual keyboard and recorded. If there is no corresponding key to be found, the recognition system will currently not try to identify the key but will carry on tracking the fingertip and detecting any further input motions. The flowchart of our virtual keyboard is shown in Figure 3. The proposed algorithms used in the virtual keyboard are to locate the virtual keyboard (Algorithm I) and extract the fingertip position in each frame (Algorithm II).

The focus and goal of our experiments are to test the virtual keyboard's performance under three main types: (1) normal typing; (2) differently customized virtual keyboards; (3) various environments and obstructions [42].

IV. PROTOTYPE IMPLEMENTATION

In this paper, our experimental equipment is a Dell Inspiron laptop and a SAMSUNG A5 smartphone. The smartphone-captured videos having finger motions were delivered to the laptop for processing. The prototype of this virtual keyboard was developed and implemented in Matlab R2014a.

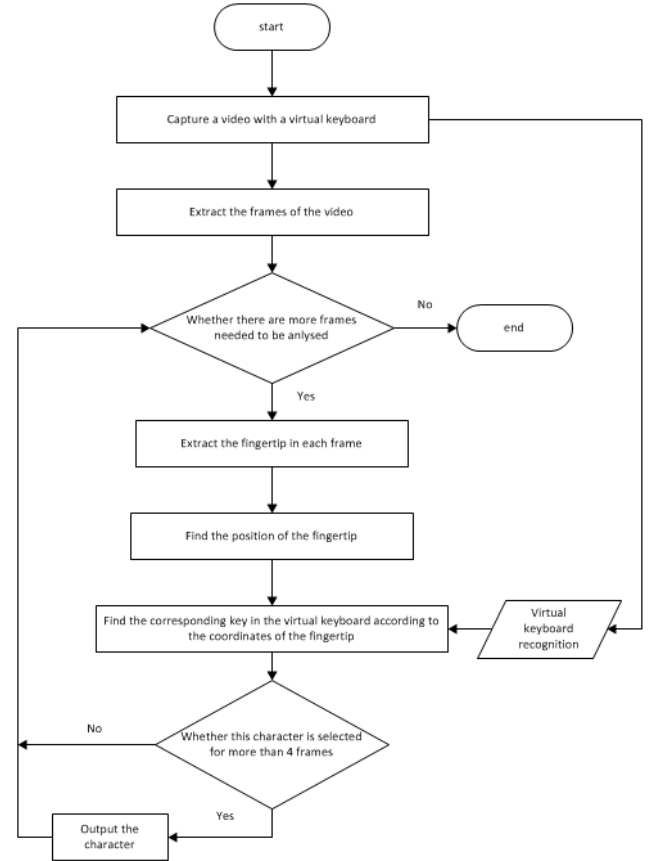


Figure 3. The flowchart of the virtual keyboard based input

Algorithm I. Locate the virtual keyboard

Input: I – keyboard background

Output: Box – areas of all characters in the background

Procedure:

Begin:

```

Results – apply OCR on the image of I
str – the string which is extracted from results without whitespace and irrelevant characters
len – length of str
box – Array with length of len to store the characters in the order according to their occurrence in the keyboard image
v = horizontal coordinate of fingertip
m = vertical coordinate of fingertip
h1 = left coordinate of the area of a character
h2 = right coordinate of the area of a character
h3 = top coordinate of the area of a character
h4 = bottom coordinate of the area of a character
if(v>h1 && v<h2 && m>h3&&m<h4){
    output this character as a candidate of user selection
}
  
```

End

Algorithm II. Extract the fingertip position in a frame

Input: *Img* – An input video frame

Output: *Img_{out}* – the video frame with hint for fingertip;

y – horizontal position of fingertip;

m – vertical position of fingertip;

Procedure:

Begin

Set *ImgPre* – Apply an Gaussian filter on *Img* to reduce noise

Set *ImgPrer* – YCbCr Image of *ImgPre*

Set *lx* – horizontal coordinate of *Img*

Set *ly* – vertical coordinate of *Img*

Set *I* – Empty Array with same size of *Img*

Set *Filter* – filter of selecting hand by setting range limit of YCbCr color value

Set *I* – Apply Filter on *ImgPrer*

Set *I* – dilate *I*

Set *I* – erode *I* to avoid noise and refining result of color filter

Set *I'* – choosing largest selected part to be human hand in *I*

Set *BW3* – using *bwmorph* to extract skeleton of *I* by using parameter of thin

Set *endPoints* – all endpoints of skeleton

Set *endpoint* – the furthest endpoint of all endpoints as fingertip

Set *v* – horizontal coordinate of endpoint

Set *m* – vertical coordinate of endpoint

End

We collected the data from three participants. Five kinds of customized virtual keyboards were selected to test the adaptability of virtual keyboard. All paper keyboards were stuck on a cream colored wall. The content of our tests was to ask our participants to type 10 Arabic numbers and 26 English letters on the chosen paper keyboards, adding up to a total of 36 keys. Testers were required to type the 36 keys in 20 seconds, which is the average speed of normal typing. This was carried out for each type of keyboard.

Five kinds of customized virtual keyboards were designed to test the adaptability of the virtual keyboard. For the sake of simplicity, we hereby refer to these keyboards as keyboard (a), keyboard (b), keyboard (c), keyboard (d) and keyboard (e), all of which are shown in Figure 4.

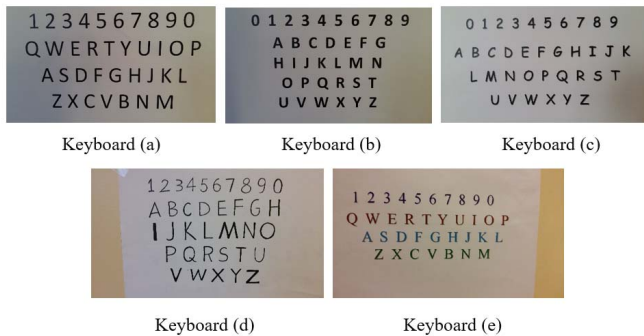


Figure 4. Customized virtual keyboards for testing

In order to test the adaptability of this virtual keyboard to various environments, we selected two typical cases for our experiments. One was under lamplight in a bedroom of a house and another was under natural light in an office room of our university. Secondly, we selected four types of customized paper keyboards to test whether our virtual keyboard is adaptive to various customized keyboards. Keyboard (a) stands for the standard keyboard and used the Calibri font. Keyboard (b) is for natural numbers and letters with a smaller font size.

Keyboard (c) was utilized to test for the adaptability to a various fonts such as Comic Sans MS. Keyboard (d) was adopted to test for adaptability to handwriting. Finally, keyboard (e) was chosen to test for the robustness of our virtual keyboard for multiple colors and obstructions. The examples of typing recognition in our experiments are shown in Figure 5.

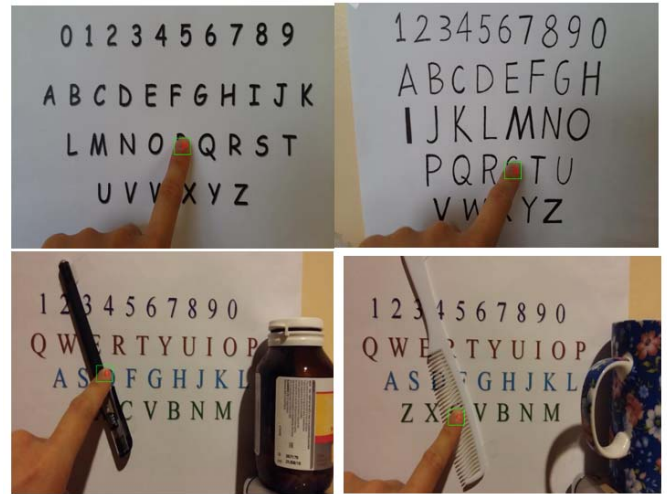


Figure 5. Examples of typing recognition

V. RESULTS

The overall recognition rate (true positives divided by all samples) of all inputs is 94.62%. The best average input recognition rate is at 97.7% from keyboard (a) under natural lighting condition, and the worst case is 90.7% from keyboard (b) under lamplight. We find that the input recognition rate is related to both the accuracy of each typing position and light source. The test results show that the robustness of our virtual keyboard for multiple colors and obstructions is excellent because the program has already recognized and recorded the entire paper keyboard before the partial occlusion occurred [1]. The test results of our input recognition of four keyboards under two different environments are shown in Table I and the results of obstruction tests are shown in Table II.

TABLE I. RESULTS OF FOUR VIRTUAL KEYBOARDS UNDER TWO ENVIRONMENTS

Keyboard	Average Recognition Rate	
	Under Lamplight	Under Natural Light
A	94.9%	97.7%
B	90.7%	93.5%
C	94.0%	95.3%
D	93.9%	97.2%

The average recognition rate under lamplight is 93.6% while the average recognition rate under natural light is 95.9%. These results show that the average recognition rate under natural light is 2.3%, higher than the rate under lamplight. This is due to our algorithm recognizing the fingertip and the 36 keys based on the colors in each frame. Thus, the lamplight makes the image of our paper keyboards dimmer than that under natural light.

TABLE II. RESULTS OF VIRTUAL KEYBOARD (E) WITH OBSTRUCTIONS

Obstructions	Mismatched Keys	Accuracy
Pen and bottle	0, 1	94.4%
Pen and bottle	1, Q	94.4%
Pen and bottle	0, 1, 2, S	88.9%
Comb and cup	N/A	100%
Comb and cup	1, 3, I	91.6%
Comb and cup	2	97.2%
The average accuracy is 94.4%		

In addition, the lamplight causes more shadows than natural light. These shadows are darker under lamplight; hence, the algorithms might treat these shadows as a part of a hand. Natural light contributes fewer shadows, which lead to less interference with the recognition rate. Thus, natural lighting has a slightly more positive influence on the performance of our virtual keyboard.

Moreover, we noticed that the recognition rate of letter ‘I’ was the lowest one when compared to the rate of other keys. The reason is that ‘I’ is quite thin, which means that the specified region of this letter is not so wide. When a user tries to type ‘I’, (s)he might not be able to precisely place the fingertip within the expected area of ‘I’. Therefore, the system will not recognize it as an effective input.

VI. CONCLUSION AND FUTURE WORK

Our contributions of this paper are expressed in four aspects. Firstly, our virtual keyboard can be printed on a plain paper or projected on a wall or other flat surface. This makes the virtual keyboard environmentally friendly because of its minimal resource use. Secondly, our virtual keyboard can be placed on any plane. In addition, when combined with 3D projection technology in the near future, it will also be possible for our virtual keyboard to be projected in mid-air. Thirdly, our users can expediently customize their own virtual keyboard. Since the virtual keyboard accepts handwritten keys, our users can create keyboards using only a pen and paper in DIY. The size and order of letters can also be designed according to the user’s needs and preferences. Finally, our virtual keyboard is simple and portable because it can be implemented simply only using a normal camera and a paper keyboard.

The experiments were designed to test the adaptability of our virtual keyboard to a plurality of environments and customized keyboards. Five types of customized paper keyboards were used under lamplight in the bedroom of a house and under natural light in an office of our university. The overall recognition rate in our tests was 94.62% and our prototype virtual keyboard therefore demonstrates proof of concept.

The focus of our future work [42] is on improving the limitations of the virtual keyboard at present through further experimentation in various contexts and with more users. The anticipated future improvements are as follows: (1) The virtual

keyboard can work in real time; (2) The prototype can identify two hands typing simultaneously; (3) It can work in dimly lit environments; (4) Other function keys such as enter, delete, shift, etc. can be added.

One limitation of our approach is the need for more sophisticated software on a separate machine, such as the image processing functions of Matlab. However, with apps growing in complexity, it may be possible for all such functions to be contained on the mobile device itself in the near future. In addition, if other machine learning technologies [33] like HMM, ANN, CNN or RNN can be incorporated into our virtual keyboard to add predictive capabilities and enhanced character recognition through dynamic gesture analysis, the recognition rate is expected to improve further [2, 41].

Another limitation of our work is the small number of users used in this feasibility study. Any larger-scale study will need to ensure that users are consistent in their keyboard skills to allow for generalizations to the user population as a whole, and this will require a separate study.

Finally, 3D projection technology will likely be more popular and less costly in the near future. In future, our virtual keyboard could be combined with 3D projection technology to project a virtual keyboard in mid-air [26, 42] where the virtual keys depress when a finger is located over them.

REFERENCES

- [1] Abate, A. F., Narducci, F., & Ricciardi, S. “An image based approach to hand occlusions in mixed reality environments”, *International Conference on Virtual, Augmented and Mixed Reality*, 2014, pp. 319-328.
- [2] Athitsos, V., Wang, H., & Stefan, A. “A database-based framework for gesture recognition”, *Personal and Ubiquitous Computing*, 14(6), 511-526, 2010.
- [3] Bérci, N., & Szolgay, P. “Towards a gesture based human-machine interface: Fast 3D tracking of the human fingers on high speed smart camera computers”, *IEEE International Symposium on Circuits and Systems*, 2009, pp. 1217-1220.
- [4] Billon, R., Nedelec, A., & Tisseau, J. “Gesture recognition in flow based on PCA analysis using multiagent system”, *International Conference on Advances in Computer Entertainment Technology*, 2008, pp. 139-146.
- [5] Bonner, M. N., Brudvik, J. T., Abowd, G. D., & Edwards, W. K. “No-look notes: accessible eyes-free multi-touch text entry”, *International Conference on Pervasive Computing*, 2010, pp. 409-426.
- [6] Chen, X., & Koskela, M. “Online RGB-D gesture recognition with extreme learning machines”, *ACM International Conference on Multimodal Interaction*, 2013, pp. 467-474.
- [7] Cheng, H., Luo, J., & Chen, X. “A windowed dynamic time warping approach for 3D continuous hand gesture recognition”, *IEEE International Conference on Multimedia and Expo*, 2014, pp.1-6.
- [8] Gadea, C., Ionescu, B., Ionescu, D., Islam, S., & Solomon, B. “Finger-based gesture control of a collaborative online workspace”, *IEEE International Symposium on Applied Computational Intelligence and Informatics*, 2012, pp. 41-46.
- [9] Gao, J., Fan, W., Jiang, J., & Han, J. “Knowledge transfer via multiple model local structure mapping”, *ACM International Conference on Knowledge Discovery and Data mining*, 2008, pp. 283-291.
- [10] Hsieh, C. C., Liou, D. H., & Lee, D. “A real-time hand gesture recognition system using motion history image”, *International Conference on Signal Processing Systems*, 2010, pp. V2-394.
- [11] Ishijima, R., Ogawa, K., Higuchi, M., & Komuro, T. “Real-time typing action detection in a 3D pointing gesture interface”, *Augmented Human International Conference*, 2014, pp. 20.

- [12] Kristensson, P. O., Nicholson, T., & Quigley, A. "Continuous recognition of one-handed and two-handed gestures using 3D full-body motion tracking sensors", ACM International Conference on Intelligent User Interfaces, 2012, pp. 89-92.
- [13] Kurakin, A., Zhang, Z., & Liu, Z. "A real-time system for dynamic hand gesture recognition with a depth sensor", European Signal Processing Conference, 2012, pp. 1975-1979.
- [14] Lee, U., & Tanaka, J. "Finger identification and hand gesture recognition techniques for natural user interface", Asia Pacific Conference on Computer Human Interaction, 2013, pp. 274-279.
- [15] Markussen, A. Jakobsen, M., and Hornbaek, K. "election-based mid-air text entry on large displays", INTERACT, 2013, pp. 401-418.
- [16] Murakami, K., & Taguchi, H. "Gesture recognition using recurrent neural networks", ACM Conference on Human Factors in Computing Systems, 1991, pp. 237-242.
- [17] Murthy, R., & Jadon, R. S. "Hand gesture recognition using neural networks", International Advance Computing Conference, 2010, pp. 134-138.
- [18] Nandakumar, K., Wan, K. W., Chan, S. M. A., Ng, W. Z. T., Wang, J. G., & Yau, W. Y. "A multi-modal gesture recognition system using audio, video, and skeletal joint data", ACM International Conference on Multimodal Interaction, 2013, pp. 475-482.
- [19] Ni, T., Bowman, D., North, C. "AirStroke: bringing unistroke text entry to freehand gesture interfaces", ACM Conference on Human Factors in Computing Systems, 2011, pp. 2473-2476.
- [20] Niikura, T., Watanabe, Y., Komuro, T., & Ishikawa, M. "In-air Typing Interface: Realizing 3D operation for mobile devices", IEEE Global Conference on Consumer Electronics, 2012, pp. 223-227.
- [21] Nikhil, S., Mohan, S., Ramya, B., & Kadambi, G. R. "Design and development of a DSP processor based reconfigurable hand gesture recognition system for real-time applications", International Conference on Signal and Image Processing, 2010, pp. 39-44.
- [22] Panwar, M. "Hand gesture recognition based on shape parameters", International Conference on Computing Communication and Applications, 2012, pp.1-6.
- [23] Prasad, J. S., Saxena, A., Javar, N., Kaushik, K. B., Chakraborty, P., & Nandi, G. C. "Gesture recognition by stereo vision. International Conference on Intelligent Interactive Technologies and Multimedia", 2010, pp. 155-162.
- [24] Raheja, J. L., Shyam, R., Kumar, U., & Prasad, P. B. "Real-time robotic hand control using hand gestures", International Conference on Machine Learning and Computing, 2010, pp. 12-16.
- [25] Sagara, S., Higuchi, M., & Komuro, T. "Multi-finger AR typing interface for mobile devices", International Conference on Intelligent User Interfaces, 2014, pp. 13-16.
- [26] Shoemaker, G., Findlater, L., Dawson, J. Q., & Booth, K. S. "Mid-air text input techniques for very large wall displays", Graphics Interface, 2009, pp. 231-238.
- [27] Sohn, M. K., Lee, S. H., Kim, D. J., Kim, B., & Kim, H. "3D hand gesture recognition from one example", IEEE International Conference on Consumer Electronics, 2013, pp. 171-172.
- [28] Song, Y., Demirdjian, D., & Davis, R. "Continuous body and hand gesture recognition for natural human-computer interaction", ACM Transactions on Interactive Intelligent Systems, 2(1), 5, 2012.
- [29] Stergiopoulou, E., & Papamarkos, N. "Hand gesture recognition using a neural network shape fitting technique", Engineering Applications of Artificial Intelligence, 22(8), 1141-1158, 2009.
- [30] Suarez, J., & Murphy, R. R. "Hand gesture recognition with depth images: A review", In RO-MAN, 2012, pp. 411-417.
- [31] Tam, V., & Li, L. S. "Integrating the Kinect camera, gesture recognition and mobile devices for interactive discussion", IEEE International Conference on Teaching, Assessment and Learning for Engineering, 2012, pp. H4C-11.
- [32] Terajima, K., Komuro, T., & Ishikawa, M. "Fast finger tracking system for in-air typing interface", In CHI'09 Extended Abstracts on Human Factors in Computing Systems, 2009, pp. 3739-3744.
- [33] Trigueiros, P., Ribeiro, F., & Reis, L. P. "A comparison of machine learning algorithms applied to hand gesture recognition", Iberian Conference on Information Systems and Technologies, 2012, pp. 1-6.
- [34] Wu, C. H., & Lin, C. H. "Depth-based hand gesture recognition for home appliance control", IEEE International Symposium on Consumer Electronics, 2013, pp. 279-280.
- [35] Wu, J., Cheng, J., Zhao, C., & Lu, H. "Fusing multi-modal features for gesture recognition", ACM International Conference on Multimodal Interaction, 2013, pp. 453-460.
- [36] Wu, J., Pan, G., Zhang, D., Qi, G., & Li, S. "Gesture recognition with a 3D accelerometer", In Ubiquitous Intelligence and Computing, 2009, pp. 25-38.
- [37] Xu, D. "A neural network approach for hand gesture recognition in virtual reality driving training system of SPG", International Conference on Pattern Recognition, 2006, 3, pp. 519-522.
- [38] Yang, J., Xu, J., Li, M., Zhang, D., & Wang, C. "A real-time command system based on hand gesture recognition", International Conference on Natural Computation, 2011, 3, pp. 1588-1592.
- [39] Zeng, B., Wang, G., & Lin, X. "A hand gesture based interactive presentation system utilizing heterogeneous cameras", Tsinghua Science and Technology, 17(3), 329-336, 2012.
- [40] Zhai, S. and Kristensson, P. "The word-gesture keyboard: reimagining keyboard interaction", Communication of the ACM, 55 (9), 91-101, 2012.
- [41] Zhang, T., & Feng, Z. "Dynamic gesture recognition based on fusing frame images", International Conference on Intelligent Systems Design and Engineering Applications, 2013, pp. 280-283.
- [42] Zhang, Y. "A virtual keyboard implementation based on finger recognition", Masters thesis, Auckland University of Technology, New Zealand, 2016.